INSTITUTO FEDERAL
Goiás

Deep Learning

Redes Neurais Artificiais: Aprendizado e Fundamentação

Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

INSTITUTO FEDERAL
Goiás

Deep Learning

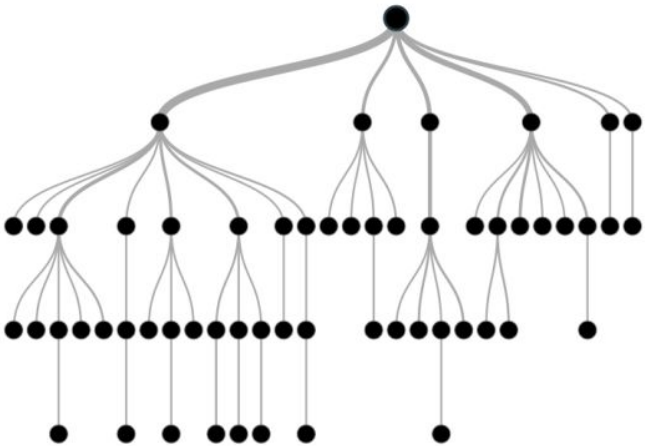
Materiais (edição 2026/2)

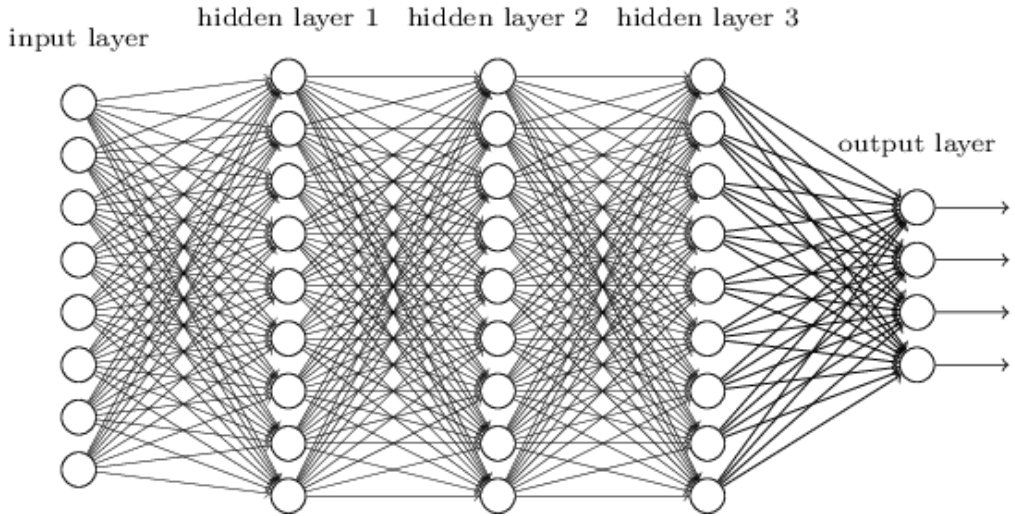
https://drive.google.com/drive/folders/1Ld3ua4GEAJKTrlK6I8kSVy0GyEG11iP9?usp=drive_link

Aulas (quinta, 19h):
https://meet.google.com/tmp-dupy-dib?authuser=0

Relembrando… O que é Deep Learning?

- If we draw a graph showing how these concepts are built on top of each other, the graph is deep, with many layers. For this reason, we call this approach to AI deep learning.





Ian J. Goodfellow, Yoshua Bengio and Aaron Courville. **Deep Learning**, MIT Press, 2016.

Inspiração: Neurônios Biológicos

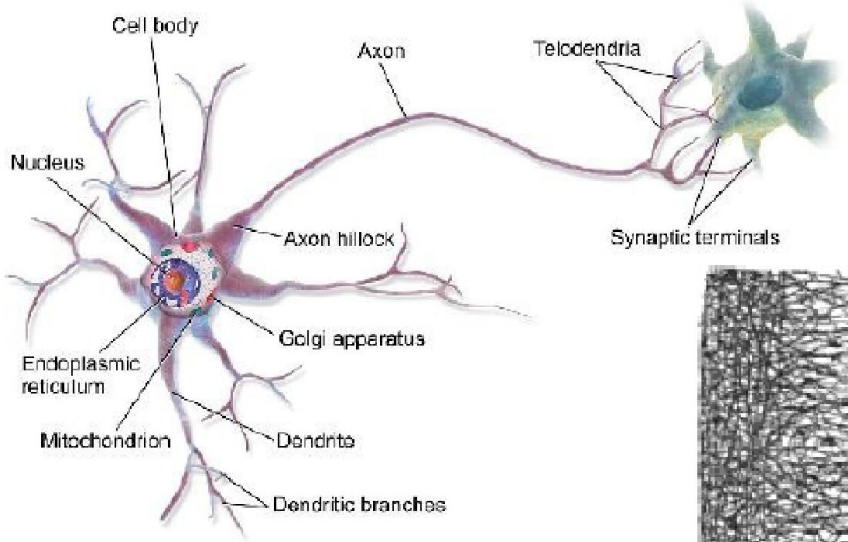


Figure 10-1. Biological neuron[3]

Conhecimento global é resultado de mudanças locais nos neurônios: *sinapses*.

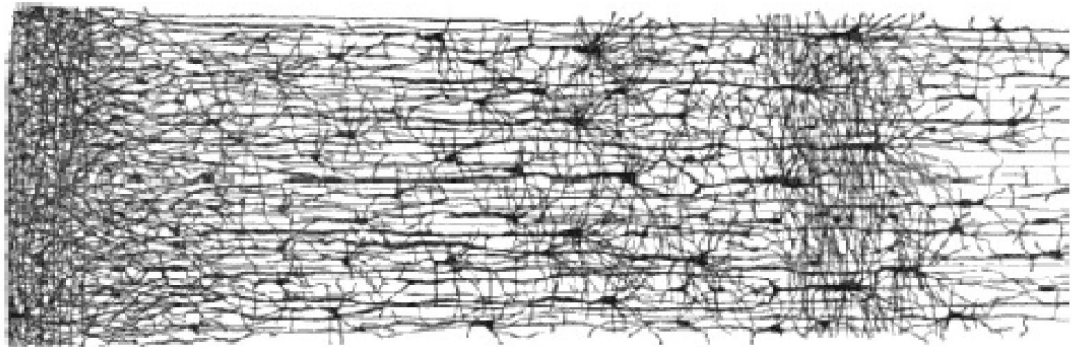


Figure 10-2. Multiple layers in a biological neural network (human cortex)[5]

Neurônio biológico modelado como capacitor

… cuja capacitância muda com a corrente recebida no tempo.

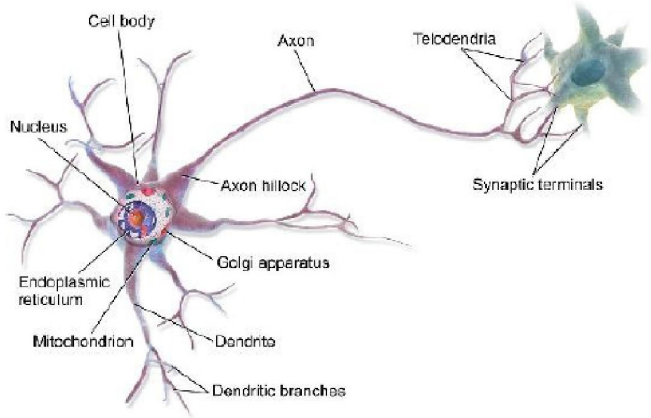
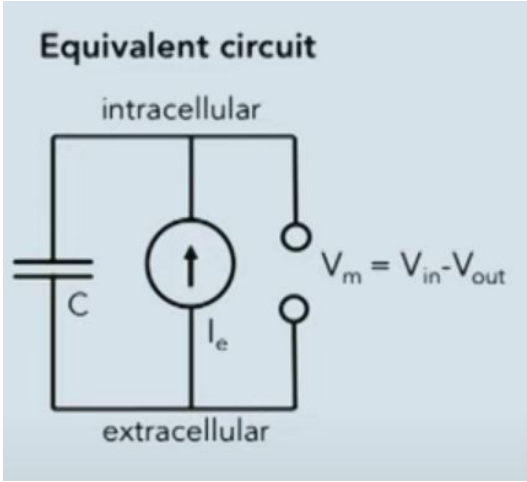


Figure 10-1. Biological neuron[3]

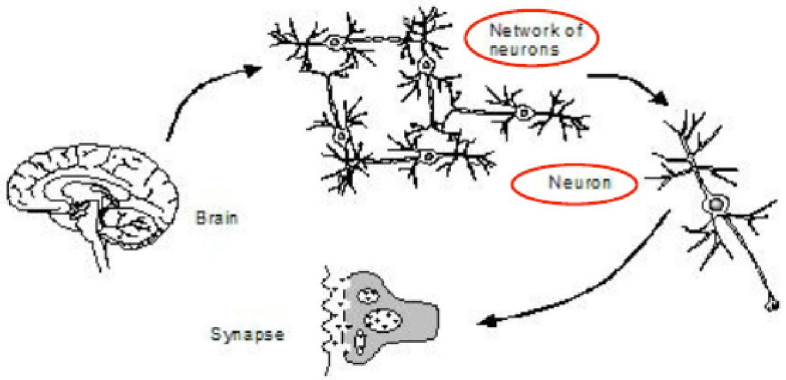
Vm (potencial elétrico da membrana):
Vin (potencial dentro do neurônio) -
Vout (potencial fora do neurônio)



Fonte: Computação Neural https://ocw.mit.edu/9-40S18

O Sistema Nervoso (SN)

- O sistema nervoso **traz informações** para o organismo sobre o ambiente ao seu redor, através de entradas sensoriais, **processa a informação** de entrada, **compara com experiências anteriores**, e **transforma** essas entradas em ações ou memória.



- Como o SN faz isso?
  - Pesquisas em neurociências em andamento…
  - Problema muito difícil!
    "Whole Brain Emulation: No Progress on C. elgans After 10 Years". *LessWrong*. 1 October 2021.

Redes Neurais Artificiais

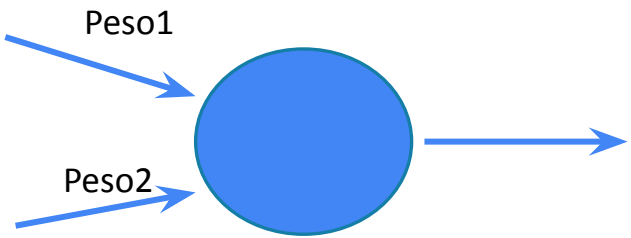
Redes Neurais Artificiais

"**Redes Neurais Artificiais** são dispositivos de computação paralela que consistem de muitos **processadores interconectados**. Esses processadores são simples... Cada processador está ciente apenas dos **sinais que manda** para e **recebe** de outros processadores periodicamente"

(R. Callan, "The Essence of Neural Networks")

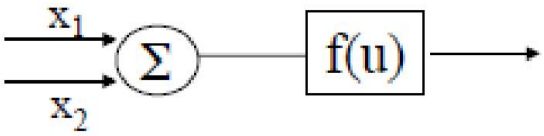
Redes Neurais: Padrão Genérico

- Caracterizadas por 3 elementos:
  - Conjunto de neurônios artificiais
  - Um padrão de conexão entre os neurônios: arquitetura ou estrutura da rede
  - Método que determina o valor dos pesos: algoritmo de treinamento ou aprendizagem



Primeiro Modelo

- “A Logical Calculus of Ideas Immanent in Nervous Activity” - McCulloch-Pitts (1943)
- Segundo eles, com um modelo simplificado é possível construir uma rede artificial de neurônios que computa qualquer proposição lógica.
- Neurônio de 2 estados
  - Comportamento de um neurônio é um processo binário
- Cada neurônio tem um limiar fixo l
  - Função de ativação:
    - Ativa a saída quando mais que um certo número de entradas estiverem ativos
  - Entradas binárias!

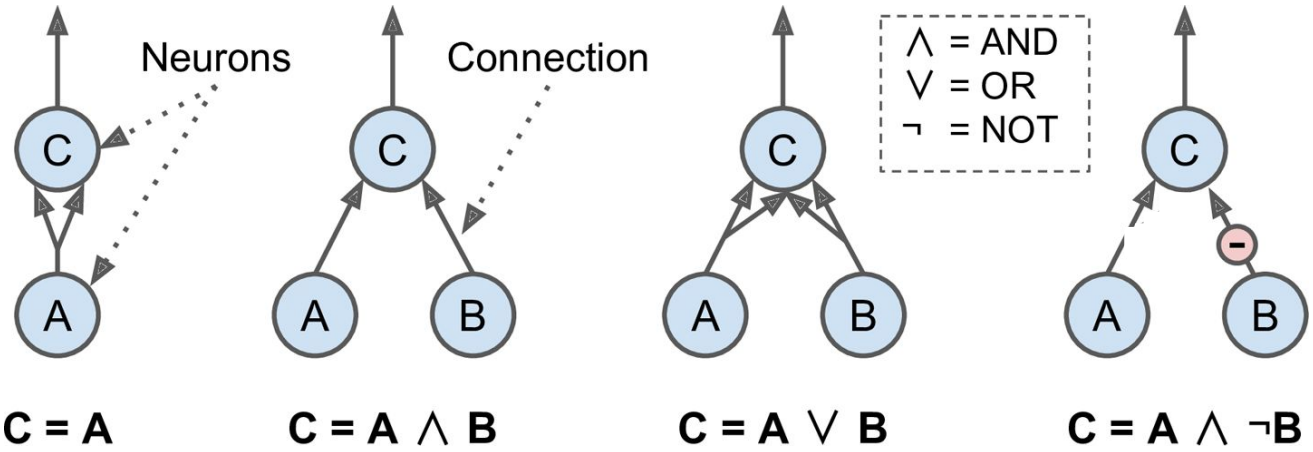


$$l = 1$$

$$l = 2$$

Primeiro Modelo

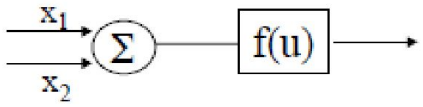
- Neurônios são “portas lógicas”
- Muito difícil “aprender” como conectar os neurônios! (sistemas digitais)



Um exemplo para Neurônio Binário

- Considere um neurônio com 2 entradas x1 e x2
- O que acontece quando o limiar:

$l = 1$



a	b	a OR b
0	0	0
0	1	1
1	0	1
1	1	1

Primeiro Modelo:

- Problema:
  - Difícil projetar a rede…
    - é mais fácil projetar um circuito digital!
    - e portanto, foi abandonado
- Entrentanto…
  - o princípio foi mantido!
    - Somatório de entradas, saídas superiores a um limiar.
- O que falta?

Segundo Modelo de neurônio: Perceptron

- Proposto por Frank Rosenblatt, 1957.
- Perceptron = “princípio do aprendizado”
- Primeiro modelo para aprendizado supervisionado com neurônios.
- Os pesos (w_1,w_2,…) são números reais que expressam a importância das respectivas entradas para as relativas saídas.

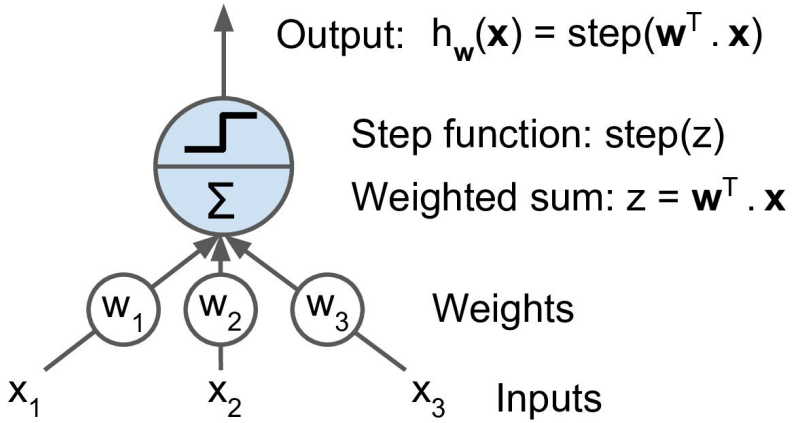
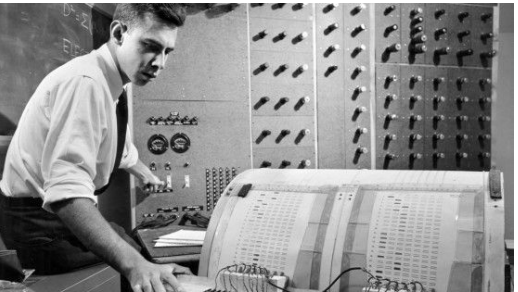


Figure 1-4. Linear threshold unit



Perceptron ou Linear Threshold Unit (LTU)

- Entradas e saídas são números, em que entradas são conectadas com pesos;
- Perceptron calcula a soma do produto dos pesos com suas entradas:
  - ($z = w_1 x_1 + w_2 x_2 + ... + w_n x_n = \mathbf{w}^T \cdot \mathbf{x}$)
- Então aplica a Step Function
  - $h_w(\mathbf{x}) = \text{step}(z) = \text{step}(\mathbf{w}^T \cdot \mathbf{x})$.

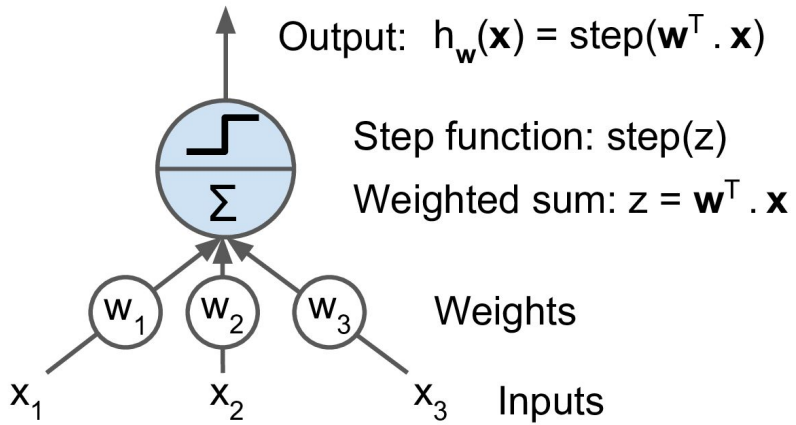


Figure 1-4. Linear threshold unit

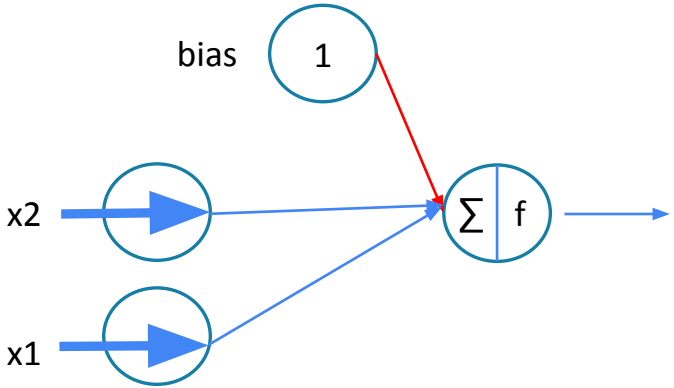
Algumas *Step Functions*:

Equation 1-1. Common step functions used in Perceptrons

$$\text{heaviside}\,(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}\,(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

Perceptron e suas partes (single-layer perceptron)

- Camada de entrada: a saída é sempre o valor recebido do atributo;
- Bias: valor adicionado aos neurônios das camadas de entrada
- Perceptron com 2 entradas e 1 saída:



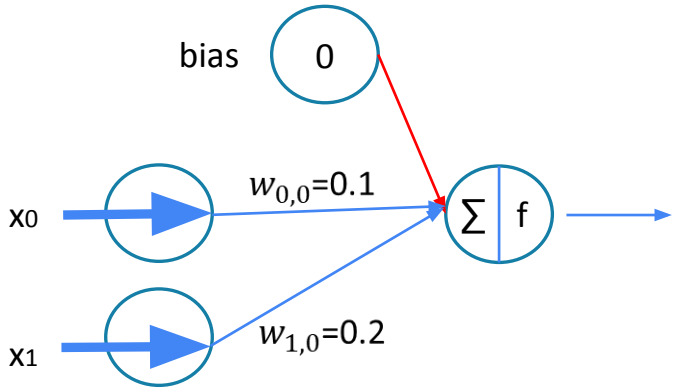
$$h_{w,b}=f(XW+b)$$

Onde:

- W são os pesos;
- b aqui é um valor de bias do neurônio;
- f é a função de ativação

Perceptron: Execução

- Dada duas entradas, x0=0.2 e x1=0.5
- Dados os pesos $w_{0,0}$=0.1 e $w_{1,0}$=0.2
- Dado o Bias=0
- Dada a função de ativação $f(x) = se\ x \geq 1; então: 1;\ senão: 0$
- Qual a saída da rede abaixo?

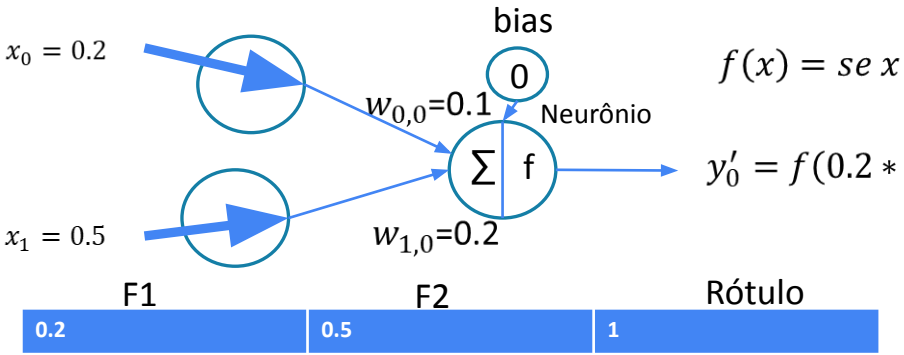


$$h_{w,b}=f(XW+b)$$

Onde:

- W são os pesos;
- b aqui é um valor de bias do neurônio;
- f é a função de ativação

Perceptron: Execução



$$f(x) = se\ x \geq 1; então: 1;\ senão: 0$$

$$y_0' = f(0.2 * 0.1 + 0.5 * 0.2 + 0) = 0$$

Contudo, a saída correta seria 1.

$w_{i,j}$ peso de conexão entre **entrada do neurônio** i e **saída do neurônio** j;

x_i valor da instancia de treino i, sendo x_0 atributo 0 e x_1 atributo 1;

y_j' saída estimada do neurônio de saída j, para a instância de treino correte;

Perceptron: treinamento

- Regra de correção do Perceptron
  - **Peso:**
    - $\mathbf{w}(t+1) = \mathbf{w}(t) + \alpha\, e_i\, x$
  - **Bias:**
    - $\mathbf{b}(t+1) = \mathbf{b}(t) + \alpha\, e_i$
  - $\boldsymbol{e_i}$ é o erro entre a saída encontrada e a desejada
  - **α** é a taxa de aprendizagem
    - No perceptron de uma camada, a taxa de aprendizagem têm pouco impacto

Perceptron: o conhecimento, parte 2

Predição(w, x, bias)

ativação = $w^T * x + bias$

retorna 1 se ativação >=0 senão 0

Repita (até convergência dos pesos)

Para cada instância (x,y)

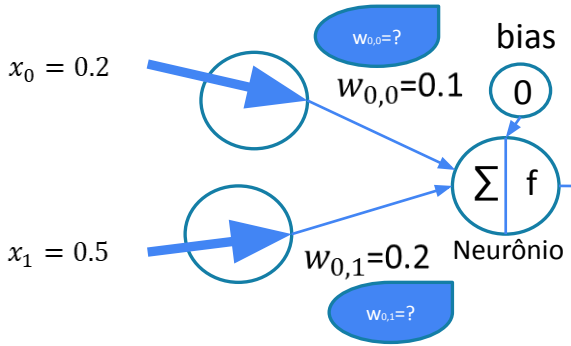
erro=y_i-predição(w, x, bias)

bias=bias + α *erro

Para cada peso w_k

w[k] = **w**[k]+ α**erro* *x[k]

Perceptron: Treinamento



$$f(x) = se\ x \geq 1; então: 1;\ senão: 0$$

$$y_0' = f(0.2 * 0.1 + 0.5 * 0.2 + 0) = f(0{,}12) = 0$$

$$w_{i,j}^{(next\ step)} = w_{i,j} + \alpha (y_j - y_j')x_i$$

0.3=0.1+1*(1-0)*0.2

0.7=0.2+1*(1-0)*0.5

$y_0' = f(0.2 * 0.3 + 0.5 * 0.7 + 0) = f(0{,}41)$=0

$$w_{i,j}^{(next\ step)} = w_{i,j} + \alpha (y_j - y_j')x_i$$

0.5=0.3+1*(1-0)*0.2

1.2=0.7+1*(1-0)*0.5

$y_0' = f(0.2 * 0.5 + 0.5 * 1.2 + 0) = f(0{,}71)$=0

$$w_{i,j}^{(next\ step)} = w_{i,j} + \alpha (y_j - y_j')x_i$$

1.0=0.5+1*(1-0)*0.2

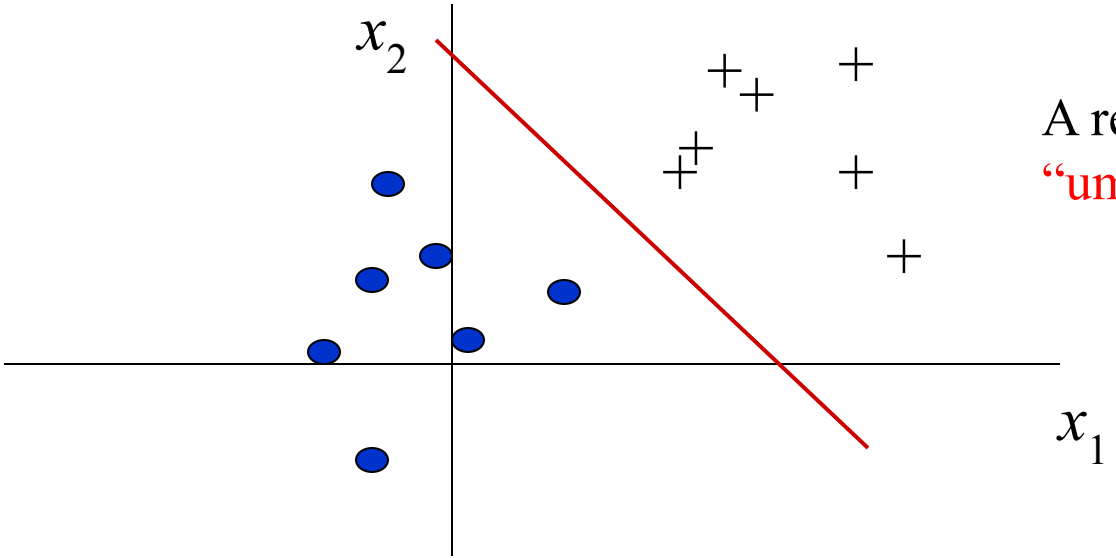
1.7=1.2+1*(1-0)*0.5

$y_0' = f(0.2 * 1.0 + 0.5 * 1.7 + 0) = f(1{,}05)$=1

Multilayer perceptron

Com um Perceptron…

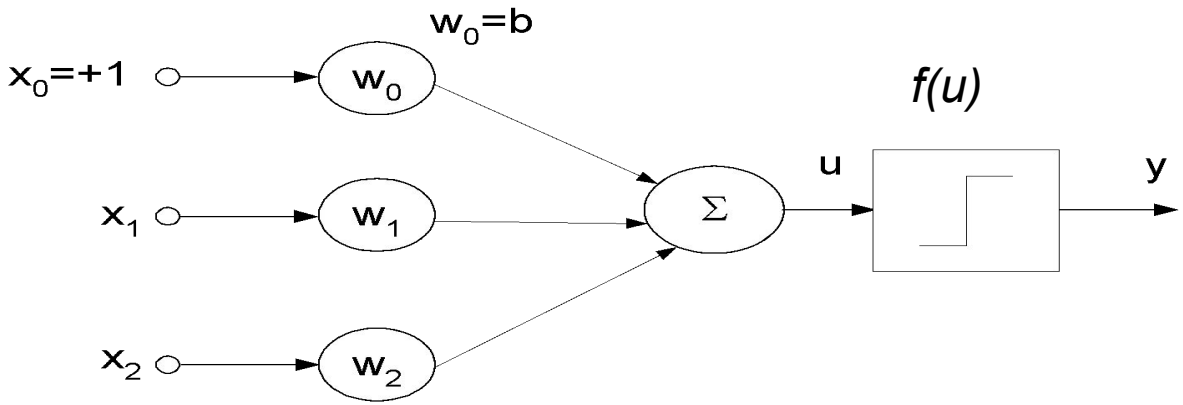
Considere, x1, x2:



A rede treinada pode ser vista como "uma linha separadora" (hiperplano).

Perceptron usado para classificação

Perceptron - O problema…



$$y = f(w_1 x_1 + w_2 x_2 + w_0), \quad \text{sendo} \quad \begin{cases} f(u) = 1 & \text{se} \quad u \geq \theta \\ f(u) = 0 & \text{se} \quad u < \theta \end{cases}$$

Com os parâmetros w_0, w_1 e w_2, a função step *f(u)* separa o espaço de entradas em duas regiões, usando uma linha reta dada por:

$$w_1 x_1 + w_2 x_2 + w_0 = \theta$$

Perceptron - O problema…

- Sabe-se que se formarmos uma combinação linear de duas variáveis, e igualá-la a um número, então os pontos no espaço bidimensional podem ser divididos em três categorias:

  – a) pontos pertencentes à linha com coordenadas tais que

  $$w_1 . x_1 + w_2 . x_2 + b = \theta$$

  – b) pontos em um lado da linha tem coordenadas tais que

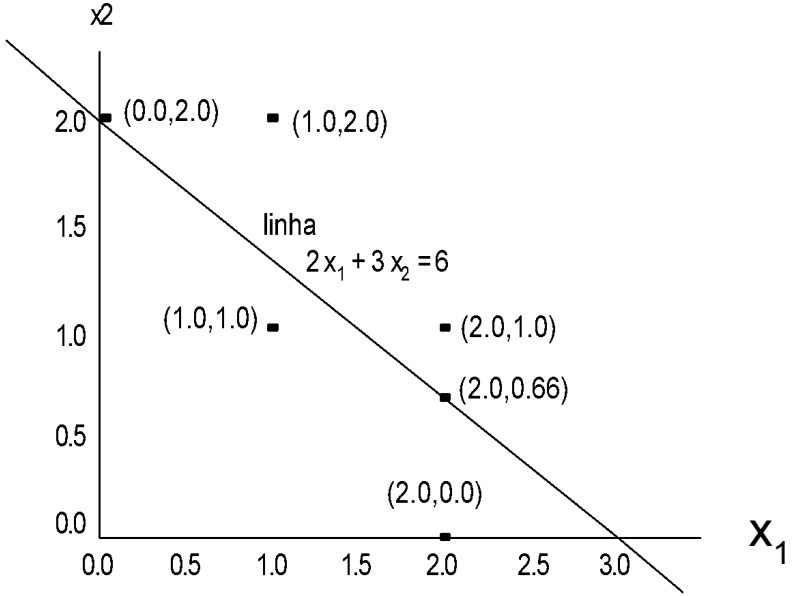
  $$w_1 . x_1 + w_2 . x_2 + b < \theta$$

  – c) pontos no outro lado da linha tem coordenadas tais que

  $$w_1 . x_1 + w_2 . x_2 + b > \theta .$$

Visualização

Pontos (x_1, x_2)	$2x_1+3x_2$	posição
(0.0, 2.0)	6	linha
(1.0, 1.0)	5	abaixo
(1.0, 2.0)	8	acima
(2.0, 0.0)	4	abaixo
(2.0, 0.66)	6	linha
(2.0, 1.0)	7	acima



Abaixo:

Posição dos pontos em função da linha $2 x_1 + 3 x_2 = 6$ de delimitação.

Linha: $2 x_1 + 3 x_2 = 6$

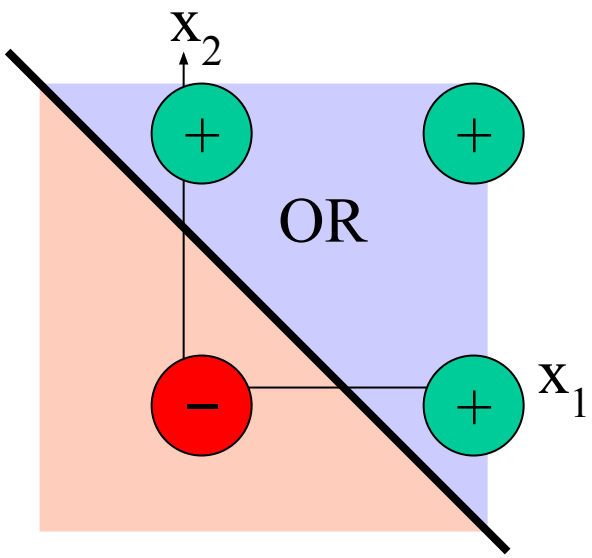
Acima: $2 x_1 + 3 x_2 > 6$

$2 x_1 + 3 x_2 < 6$

$\theta = 6$

Visualização: Exemplos de treino OR

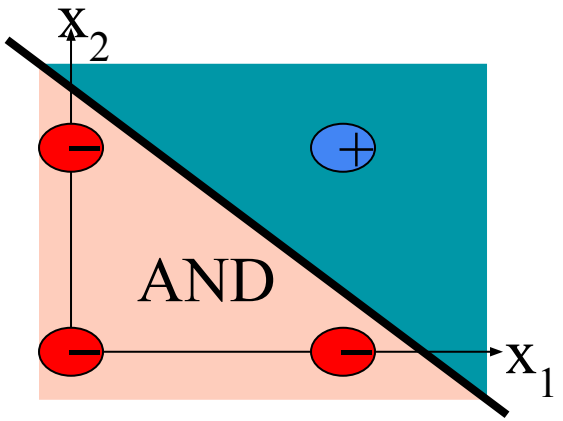
x_1	x_2	x_1+x_2
0	0	0
0	1	1
1	0	1
1	1	1
		OR



Visualização: Exemplos de Treino AND

x_1	x_2	$x_1 \cdot x_2$
0	0	0
0	1	0
1	0	0
1	1	1

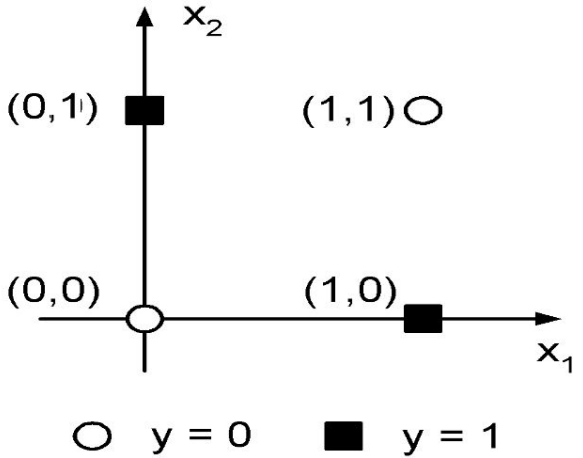
AND



Perceptron - O problema do XOR

No caso do XOR, não existe uma única reta que divide os pontos (0,0) e (1,1) para um lado, e (0,1) e (1,0) do outro lado.

x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0



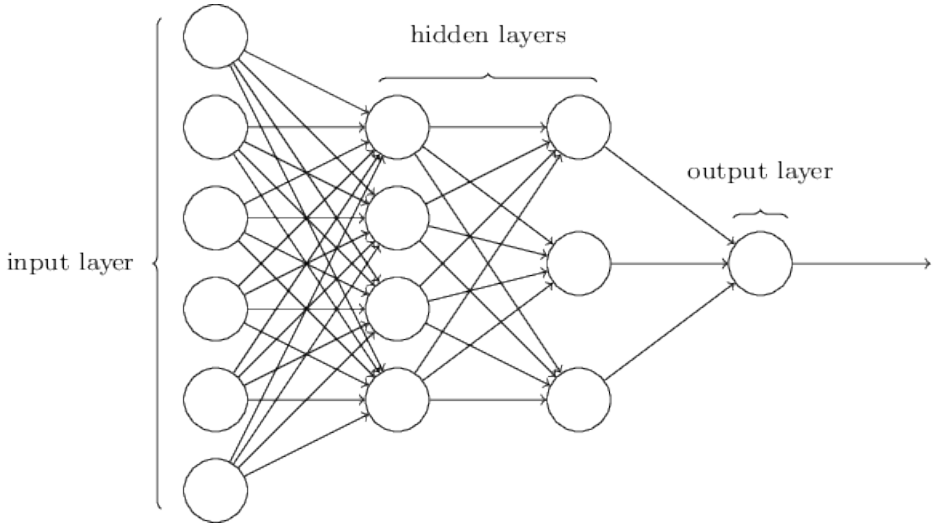
Conclui-se que um neurônio do tipo perceptron não implementa uma função ou-exclusivo (constatado por Minsky & Papert, em 1969).

A Solução…
Multilayer Perceptron

INSTITUTO FEDERAL
Goiás

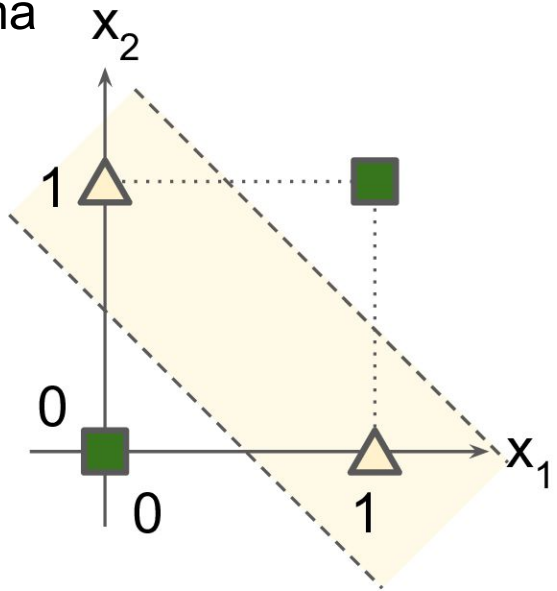
- Saída de um perceptron sendo a entrada de outro...

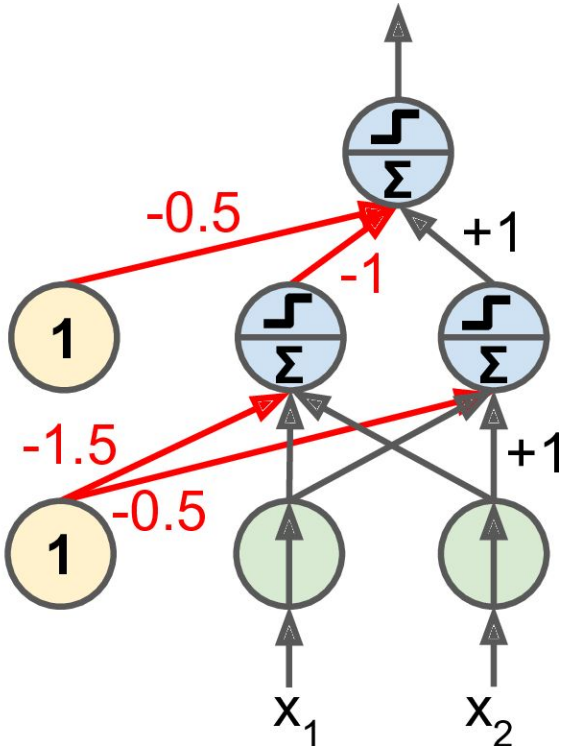
- Assim, o perceptron da segunda camada pode tomar decisões ainda mais complexas e abstratas;



A Solução…
Multilayer Perceptron

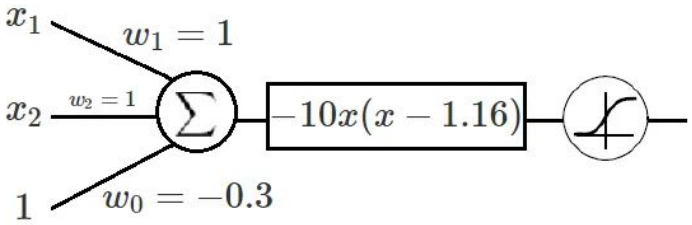
- Ao incluir ao menos uma camada escondida! (hidden layer)

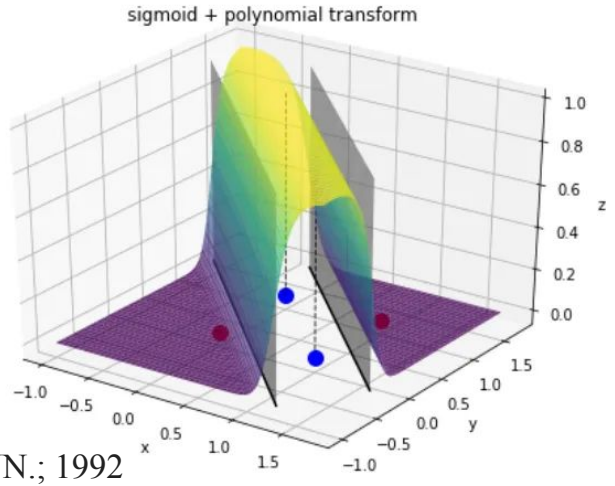




Uma Solução alternativa… com single “Perceptron alterada”

- Incluir complexidade na função de ativação.
- Similar a “funções de kernel” do SVM.
  - No caso do xor, f(x)=-10x(x-1.16)
- Na prática, não é usada!
  - Descobrir funções de ativação complexas para cada problema torna-se inviável!
  - Se fôssemos seguir por esse caminho, melhor voltar ao SVM.





Piazza, F. **Artificial Neural Networks With Adaptive Polynomial Activation Function.** IJCNN.; 1992

Atividade

- Implemente o algoritmo anterior;
- Considerando os pesos e o bias inicial iguais a zero, taxa de aprendizado = 0.1 e épocas = 5.
- Ainda, considere o seguinte conjunto de entrada:

2,781083600	2,550537003	0
1,465489372	2,362125076	0
3,396561688	4,400293529	0
1,38807019	1,850220317	0
3,06407232	3,005305973	0
7,627531214	2,759262235	1
5,332441248	2,088626775	1
6,922596716	1,77106367	1
8,675418651	-0,242068655	1
7,673756466	3,508563011	1

- O resultado esperado é:
  - Peso= [0.20653640140000007, -0.23418117710000003]
  - Bias: -0.1

Descrição da atividade: bit.ly/41IY5DX

Referências

[1] Aurélien Géron. Neural networks and deep learning (cap. 1) https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html

[2] NIELSEN, M. A. Neural Networks and Deep Learning. Determination Press, 2015. [Online]. Disponível em <http://neuralnetworksanddeeplearning.com/>

[3] GOODFELLOW, Y. BENGIO, A. COURVILLE. Deep Learning. MIT Press, 2017.